Campusmonk

PYQ Coding

229. Majority Element II

15. 3Sum

152. Maximum Product Subarray

Campusmonk

229. Majority Element II

Given an integer array of size `n`, find all elements that appear more than `⌊n / 3⌋` times.

Example 1:

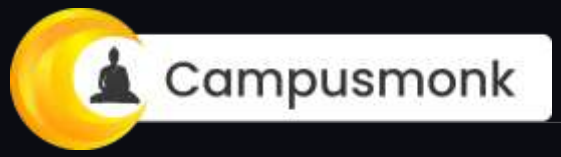
```
Input: nums = [3,2,3]
Output: [3]
```

Example 2:

```
Input: nums = [1]
Output: [1]
```

Example 3:

```
Input: nums = [1,2]
Output: [1,2]
```



Java Auto

```java
class Solution {
    public List<Integer> majorityElement(int[] nums) {
        int candidate1 = 0;
        int candidate2 = 0;
        int count1 = 0;
        int count2 = 0;
        for (int num : nums) {
            if (num == candidate1) {
                count1++;
            }
            else if (num == candidate2) {
                count2++;
            }
            else if (count1 == 0) {
                candidate1 = num;
                count1 = 1;
            }
            else if (count2 == 0) {
                candidate2 = num;
                count2 = 1;
            }
            else {
                count1--;
                count2--;
            }
        }
        count1 = 0;
        count2 = 0;
        for (int num : nums) {
            if (num == candidate1) {
                count1++;
            }
            else if (num == candidate2) {
                count2++;
            }
        }
        List<Integer> result = new ArrayList<>();
        if (count1 > nums.length / 3) {
            result.add(candidate1);
        }
        if (count2 > nums.length / 3) {
            result.add(candidate2);
        }
        return result;
    }
}
```

Campusmonk

15. 3Sum

Given an integer array nums, return all the triplets `[nums[i], nums[j], nums[k]]` such that `i != j`, `i != k`, and `j != k`, and `nums[i] + nums[j] + nums[k] == 0`.

Notice that the solution set must not contain duplicate triplets.

Example 1:

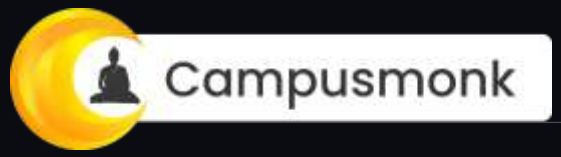
```
Input: nums = [-1,0,1,2,-1,-4]
Output: [[-1,-1,2],[-1,0,1]]
Explanation:
nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.
nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.
nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.
The distinct triplets are [-1,0,1] and [-1,-1,2].
Notice that the order of the output and the order of the
triplets does not matter.
```

Example 2:

> **Input:** nums = [0,1,1]
> **Output:** []
> **Explanation:** The only possible triplet does not sum up to 0.

Example 3:

> **Input:** nums = [0,0,0]
> **Output:** [[0,0,0]]
> **Explanation:** The only possible triplet sums up to 0.



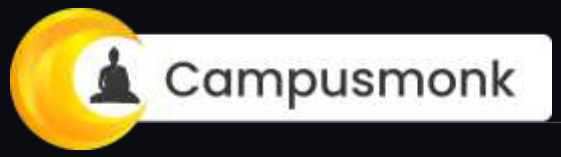

```java
class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
       List<List<Integer>> ans = new ArrayList<>();
       int n=nums.length;
       Arrays.sort(nums);

       for(int i=0;i<n-2;i++){
          if (i > 0 && nums[i] == nums[i - 1]) {
             continue;
                  }
        int j=i+1;
        int k=n-1;

        while(j<k){
            List<Integer>temp=new ArrayList<>();
            int sum=nums[i]+nums[j]+nums[k];
            if(sum==0){
                temp.add(nums[i]);
                temp.add(nums[j]);
                temp.add(nums[k]);
                ans.add(temp);

                // skip left and right elements if same pair
                while(j<k && nums[j]==nums[j+1]){
                j++;
                }
                while(j<k && nums[k]==nums[k-1]){
                k--;
                }

                j++;
                k--;
            }
            else if(sum>0){
            k--;
            }
            else if(sum<0){
            j++;
            }
        }
       }
       return ans;
    }
}
```




```cpp
vector<vector<int>> threeSum(vector<int>& nums) {
    vector<vector<int>> ans;
    int n = nums.size();
    sort(nums.begin(), nums.end());
    for (int i = 0; i < n - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1])
            continue;
        int j = i + 1;
        int k = n - 1;
        while (j < k) {
            int sum = nums[i] + nums[j] + nums[k];
            if (sum == 0) {
                ans.push_back({nums[i], nums[j], nums[k]});
                while (j < k && nums[j] == nums[j + 1])
                    j++;
                while (j < k && nums[k] == nums[k - 1])
                    k--;
                j++;
                k--;
            }
            else if (sum > 0) {
                k--;
            }
            else {
                j++;
            }
        }
    }
    return ans;
}
```

```python
def threeSum(self, nums):
    ans = []
    n = len(nums)
    nums.sort()
    for i in range(n - 2):
        # Skip duplicate first elements
        if i > 0 and nums[i] == nums[i - 1]:
            continue
        j = i + 1
        k = n - 1
        while j < k:
            total = nums[i] + nums[j] + nums[k]
            if total == 0:
                ans.append([nums[i], nums[j], nums[k]])
                # Skip duplicate left elements
                while j < k and nums[j] == nums[j + 1]:
                    j += 1
                # Skip duplicate right elements
                while j < k and nums[k] == nums[k - 1]:
                    k -= 1
                j += 1
                k -= 1
            elif total > 0:
                k -= 1
            else:
                j += 1
    return ans
```

Campusmonk

152. Maximum Product Subarray

Given an integer array `nums`, find a subarray that has the largest product, and return *the product*.

The test cases are generated so that the answer will fit in a **32-bit** integer.

Note that the product of an array with a single element is the value of that element.

Example 1:

```
Input: nums = [2,3,-2,4]
Output: 6
Explanation: [2,3] has the largest product 6.
```

Example 2:

```
Input: nums = [-2,0,-1]
Output: 0
Explanation: The result cannot be 2, because [-2,-1] is not a subarray.
```

Java Auto

```java
class Solution {
    public int maxProduct(int[] nums) {
        int n = nums.length;
        int prefix = 1;
        int suffix = 1;
        int ans = Integer.MIN_VALUE;
        for (int i = 0; i < n; i++) {
            // Reset when previous product became 0
            if (prefix == 0) {
                prefix = 1;
            }
            if (suffix == 0) {
                suffix = 1;
            }
            // Product from left
            prefix = prefix * nums[i];
            // Product from right
            suffix = suffix * nums[n - 1 - i];
            ans = Math.max(ans, Math.max(prefix, suffix));
        }
        return ans;
    }
}
```

Java Auto

```java
class Solution {
    public int maxProduct(int[] nums) {

        int maxProduct = nums[0];
        int minProduct = nums[0];
        int ans = nums[0];

        for (int i = 1; i < nums.length; i++) {

            // If current number is negative,
            // max and min will swap roles
            if (nums[i] < 0) {
                int temp = maxProduct;
                maxProduct = minProduct;
                minProduct = temp;
            }

            maxProduct = Math.max(nums[i], maxProduct * nums[i]);
            minProduct = Math.min(nums[i], minProduct * nums[i]);

            ans = Math.max(ans, maxProduct);
        }

        return ans;
    }
}
```